

Lecture 06

Nested Patterns and Tail Recursion

Defining our own option types

Using what we already know:

```
type int_option = NoInt | OneInt of int
```

Using new feature of defining our own *type constructor*

```
type 'a myoption = MyNone | MySome of 'a
```

Exactly how “built-in” options are defined (just in standard library):

```
type 'a option = None | Some of 'a
```

So use pattern-matching!

- None and Some are (just) constructors for 'a option
- *Build* with constructors and *use* with match expressions
 - Do not use Option.get anymore (!)
 - Do not use e = None anymore (!)
 - Get the advantages of variant types and pattern matching

Now lists

Can use the same features and self-reference for our own list type constructor

```
type 'a my_list = Empty | Cons of 'a * ('a my_list)
```

Build with constructors:

```
let xs = Cons(3, Cons (4, Cons (5, Empty)))  
let ss = Cons("hi", Cons("bye", Empty))
```

Use with match expressions:

```
let rec length xs =  
  match xs with  
  | Empty -> 0  
  | Cons(x,xs') -> 1 + length xs'
```

Group Work

1. Write a function `sum_int_list` that sums the values in an `int_list`
2. Write a function `sum_int_option_list` that sums the values in an `int_option mylist`

```
type int_option =  
  | NoInt  
  | OneInt of int
```

```
type 'a myoption =  
  | MyNone  
  | MySome of 'a
```

```
type int_list =  
  | IntEmpty  
  | IntCons of int * int_list
```

```
type 'a mylist =  
  | MyEmpty  
  | MyCons of 'a * 'a list
```

OCaml's built-in lists

- Could have done *exactly* what we just did
- But instead used special *syntax*
 - Constructors are `[]` and `::` with `::` written *infix*
 - Use match expressions to use lists
 - *not* `List.hd`, `List.tl` and `e=[]` anymore (!!)

```
let rec length xs =  
  match xs with  
  | [] -> 0  
  | x::xs' -> 1 + length xs'
```

Why List.hd and List.tl?

- Occasionally convenient
 - Including for passing functions to other functions (next week)
- A stepping stone to the more elegant and less error-prone approach
 - match expressions warn on missing cases!
- Again, do not use these functions on remaining homeworks
 - Easy to define:

```
let hd xs =  
  match xs with  
  | [] -> failwith "list empty"  
  | x::xs' -> x
```

```
let isEmpty xs =  
  match xs with  
  | [] -> true  
  | x::xs' -> false
```

How languages define variant types

- OCaml built powerful pattern-matching into the language
 - Used it as *the* fundamental primitive to *use* one-of types
- Other languages make other choices. OCaml *could* have:
 - Kept variant type definitions as-is
 - Had them automatically introduce *functions* like isEmpty and hd

More pattern-matching

OCaml also has pattern-matching for each-of types

- Pattern (x, y, z) matches triples (v_1, v_2, v_3)
 - Similar for any width tuple, naturally
 - Similar patterns $\{f_1=x_1; \dots; f_n=x_n\}$ for records

Useful for *nested patterns* (next lecture)

But also useful to make *let expressions more powerful*

Do NOT do this

One-branch match expressions make semantic sense but are BAD / SILLY style

```
let sum_triple tr =  
  match tr with  
  | (x,y,z) -> x+y+z
```

New feature

Let expression syntax is actually `let p = e1 in e2`

- Where `p` is a *pattern*

(Top-level let bindings can also do this)

Great for extracting multiple pieces of data at once

- No more need for `fst`, `snd`, `fst3`, `snd3`, `thd3`, ...

```
let sum_triple tr =  
  let (x,y,z) = tr in  
  x+y+z
```

Another new feature

Function arguments can also use patterns

- Function call semantics: pattern-match the argument, extracting data for body

```
let sum_triple tr =  
  let (x,y,z) = tr in  
  x+y+z
```

```
let sum_triple (x,y,z) =  
  x+y+z
```

```
let ten = sum_triple (5,2,3)
```

Hmm... 😊

Compare:

A function that takes a triple and sums its int components:

```
let sum_triple (x,y,z) =  
    x+y+z
```

A function that takes three ints and sums them:

```
let sum_triple (x,y,z) =  
    x+y+z
```

See the difference? (Me neither.) 😊 🎉 😊 🎉 😊

The truth about functions

In OCaml, every function takes exactly one argument

We can *simulate* multiple arguments with tuples and tuple-patterns

- Elegant, compositional design

Enables sometimes-convenient things

- Return a tuple from one function and pass it like multiple arguments to another

Next unit will see a different and more common way to simulate multiple arguments that *also* has built-in syntactic support

Nested Patterns

Nested patterns

- We can nest patterns inside other patterns
 - Just like we can nest expressions as deep as we want
 - Anywhere a variable can appear in our current patterns, we can put a pattern
 - Often avoids hard-to-read nested match expressions
- Full meaning of pattern-matching: compare a pattern against a value for the “same shape” and bind variables to the “right parts”
 - More precise recursive definition coming after some examples

Examples

- Match on multiple things with same shape at same time
- Go multiple levels into data structure at once
- Use wildcard patterns when you do not need the data
- Make match expressions that look like tables

Nested match expressions?

- Sometimes a match expression inside a match expression is a missed opportunity for nested pattern matching

```
match xs with  
| [] -> 0  
| x::xs' -> match xs' with ...
```

- Other times it is necessary because you need to first compute with data extracted via the outer match

```
match xs with  
| [] -> 0  
| x::xs' -> (match f xs' with ...)
```

Elegant generalization, more precisely

Syntax:

match e with $P1 \rightarrow e1 \mid \dots \mid PN \rightarrow eN$

let $P1 = e$ in $e1$

let [rec] $f \ P1 = e1$

Where

- $e, e1, \dots, eN$ are expressions
- $P1, \dots, PN$ are patterns

Patterns are **NOT** expressions!

(though they kinda look like expressions)

Syntax: A pattern is one of (syntax)

- x : a variable
- $_$: a wildcard
- $(p1, \dots, pN)$: a tuple of patterns p_i
- $C \ p$: a constructor C applied to a pattern p
- ...

Evaluation rules

(Will now focus on match expressions, others are similar)

Need two things:

1. Semantics of “matching a pattern p to a value v ”
 - Does or does not match
 - If matches, creates a set of (local) bindings
2. Use (1) to define semantics for match expressions

Pattern-matching revealed

Yet another elegant recursive definition:

Given pattern p and value v :

- If p is a variable x , the match succeeds and x is bound to v .
- If p is $_$, the match succeeds and no bindings are introduced.
- If p is $(p1, \dots, pn)$ and v is $(v1, \dots, vn)$, the match succeeds if and only if $p1$ matches $v1$, ..., pn matches vn . The bindings are the union of all bindings from the submatches.
- If p is $C\ p1$, the match succeeds if and only if v is $C\ v1$ (i.e., the same constructor) and $p1$ matches $v1$. The bindings are the bindings from the submatch.

Examples

- Pattern $\mathbf{a :: b :: c :: d}$ matches all lists with ≥ 3 elements
- Pattern $\mathbf{a :: b :: c :: []}$ matches all lists with exactly 3 elements
- Pattern $\mathbf{((a, b) , (c, d)) :: e}$ matches all non-empty lists of pairs of pairs

Semantics of match expressions

```
match e with P1 -> e1 | ... | PN -> eN
```

1. Evaluate e to v
2. Check in $P_1, P_2, \dots$ order; find first P_i that matches
3. Evaluate e_i in environment extended with bindings from the match
4. Result of (3) is the overall result

Note: This is the *semantics*, but the implementation can *optimize* with concepts like binary search

Note: Semantics for let bindings similar, with one pattern to match

Type-checking

```
match e with P1 -> e1 | ... | PN -> eN
```

- Type-check e to some type t
- Each P_i must match against some value of type t
- Similar pattern, type matching definition to get the types of bindings
 - Typecheck e_i in a static environment that has those bindings
- All $e_1, \dots, e_n$ must have the same type t_2 , which is the overall type

Also, some fantastic features:

- Warnign if a P_i can never match because of earlier patterns (dead code)
 - Warning if a value of type t could match no P_i (incomplete match)
- (With nested patterns, these require some fairly fancy algorithms)

Tail Recursion

Recursion

- Now have lots of practice with recursion
 - “Code follows the data”: recursive data processed recursively!
- No need for loops and assignment statements. Recursion often easier:
 - Example: Processing trees
 - Example: Appending lists
- But every recursive call needs to get “fresh space” for arguments and local variables, separate from the caller’s “space”

Next: refine our mental model of how calls execute (**call stacks**)

- And see how recursion can be *as efficient as a loop*!

Call Stacks

Function calls finish in the reverse order that they start



While a program runs, there is a **call stack** for all function calls that have started but not yet completed

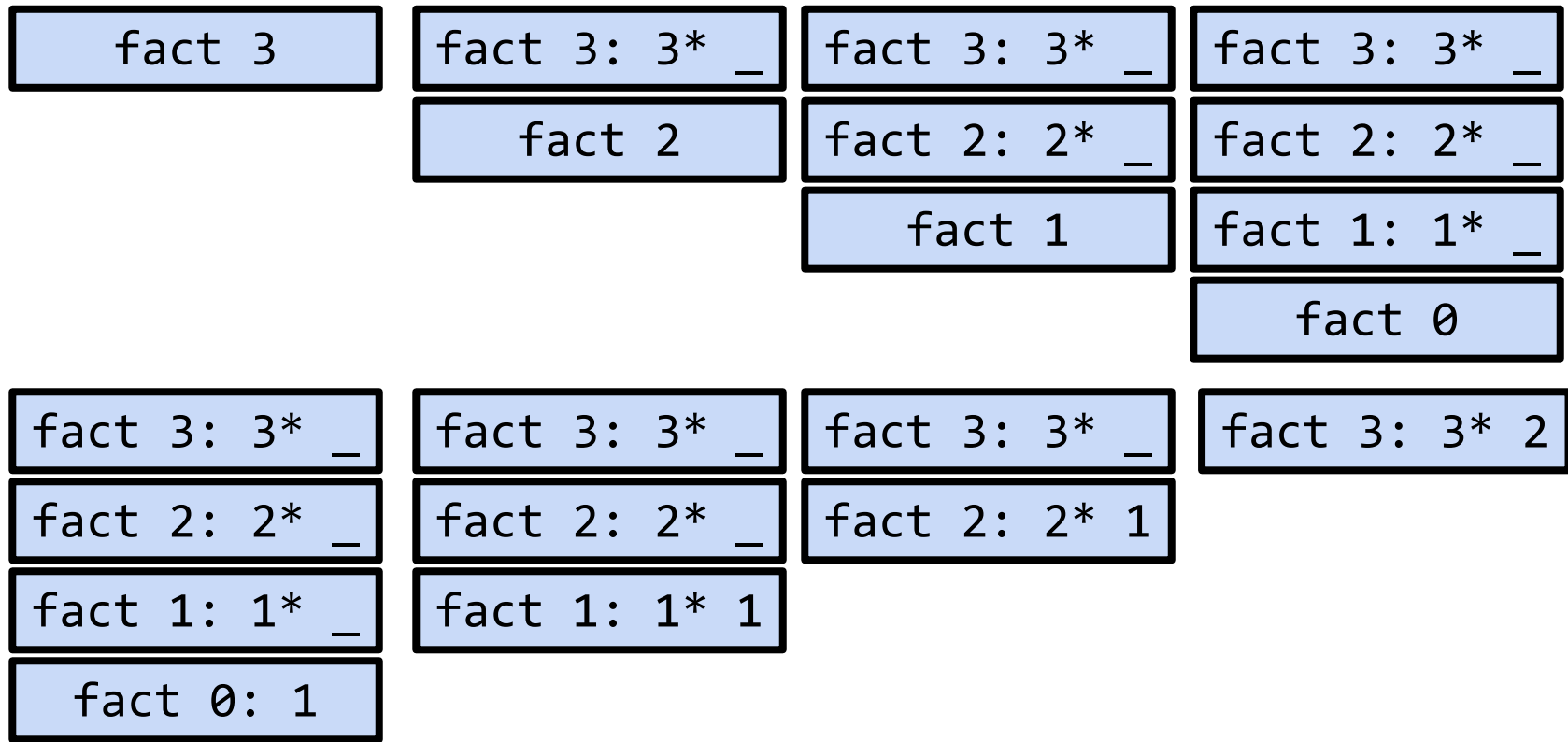
- Calling **f** pushes a **stack frame** for **f** on the stack
- When a call finishes, its stack frame is popped from the stack
- Recursion: > 1 stack frames can be for calls to the same function

A stack frame stores information: values of parameters, values of local variables, and “what’s left to do” for the function body

- Low-level details studied in CSE 351, but *concept* needn’t be low-level

Example

```
let rec fact n = if n=0 then 1 else n*fact(n-1)
let x = fact 3
```



Example

```
let rec last xs =  
  match xs with x::[] -> x | _::xs' -> last xs'  
let x = last [3;7;2]
```

last [3;7;2]

last[3;7;2]:_

last[3;7;2]:_

last [7;2]

last[7;2]: _

last [2]

last[3;7;2]:_

last[3;7;2]:_

last[3;7;2]:2

last[7;2]: _

last[7;2]: 2

last[2]: 2

Do we ALWAYS need a stack frame? (Hint: No.)

- *It is unnecessary to keep around a stack-frame just so it can get a result from a function call and immediately return it “as is”*
 - A call that will be the caller’s answer “as is” is called a **tail call**
 - A recursive function where all recursive calls are tail calls is called “tail recursive”
- OCaml handles tail calls specially: *tail-call optimization*
 - Pops caller’s frame **BEFORE** the call
 - This can take $O(N)$ space usage down to $O(1)$ 🤖
 - “No stack overflow”
 - Along with other optimizations, as efficient as a loop

last is tail-recursive!

```
let last xs =  
  match xs with x::[] -> x | _::xs' -> last xs'  
  
let x = last [3;7;2]
```

last [3;7;2]

last [7;2]

last [2]

Making functions tail-recursive

Sometimes we can rewrite our algorithm to be tail-recursive

```
let fact n =  
  let rec loop (n,acc) = if n=0 then acc else loop (n-1,acc*n) in  
  loop (n,1)  
let x = fact 3
```

Sometimes we can't easily/profitably do so:

- Example: processing a tree (two recursive calls can't both be tail calls)
- Example: if order matters (e.g., concatenating a string list)

Methodology

There is a *methodology* to guide the transformation to a tail-recursive function:

- Create a helper function that takes an *accumulator*
- Old base case becomes initial accumulator
- New base case becomes final accumulator

Up to you to reason about “whether this works”

- Example: for factorial, it is crucial that “multiplication is associative”

More examples (see code)

- Summing a list: $O(1)$ stack space vs. $O(n)$ stack space (n = length of list)
 - But notice both versions of factorial and list-sum take $O(n)$ *time*
- Reversing a list: Tail-recursive version actually *much better*
 - Non-tail recursive version is *quadratic time*
 - Each recursive call uses `append`, which must traverse the list
 - So total time for the appends is $1+2+\dots+(\text{length}+1)$
 - roughly $\text{length} * \text{length} / 2$
 - Moral: **beware list append**, especially within outer recursion
 - `Cons` is constant time and fast, so tail-recursive version is linear time

Moral / Don't Overdo It!

- Tail-recursion is not always feasible or elegant
- Tail-recursion is not always necessary
 - Beware premature optimization and favor readability
- But where it is feasible and efficiency matters, it is a key tool for functional programmers to get the efficiency of loops
 - If you can write it as a loop, you can write it as a tail-recursive function -- “updated values” are arguments to the tail call!
- Most functional languages promise tail-call optimization

Precise definition of tail calls

How do you know if a call is a tail call?

Humans: “Oh, I see the caller has no more work to do after it gets the result.”

Semanticists and compilers:

- A tail call is a function call that “is in tail position”
- “is in tail position” is defined recursively over the syntax of expressions...

Tail position

- In **let** **f** **p** = **e**, the expression **e** is in tail position
- If **if** **e1** **then** **e2** **else** **e3** is in tail position
 - Then **e2** and **e3** are in tail position
 - BUT **e1** is NOT in tail position!
- If **let** **p** = **e1** **in** **e2** is in tail position
 - Then **e2** is in tail position
 - BUT **e1** is NOT in tail position
- *The expression args to a function call are NOT in tail position*
- For **e1** + **e2**, the subexpressions are NOT in tail position
- ... (more cases for every kind of expression)
- If **e** is NOT in tail position, then neither are any of its subexpressions

Group Work

```
let rec length (l : 'a list) =  
  match l with  
  | [] -> 0  
  | hd :: tl -> 1 + length tl
```

1. Is this function tail recursive?
2. If it is not tail recursive, write a version of the function that is tail recursive